

# FAHM PROJECT

## Project Understanding and Scientific Practice Questionnaire

---

Current-stage review, scientific practice questionnaire, and alignment with a predictive-maintenance Machine Learning Engineer role

### **Purpose**

Build a professional and scientific understanding of the work already completed, identify the project's main methodological risks, practice explaining the reasoning behind each decision, and connect the project roadmap directly to predictive-maintenance Machine Learning Engineer requirements.

Project owner: Leonardo Romano

Prepared July 2026

## How to Use This Guide

- Read the project summary and scientific risks before answering the questions.
- For guided examples, first attempt your own answer, then compare it with the model answer.
- For practice questions, write concise answers in your own words and support each claim with project evidence.
- Revisit this guide after each new notebook or project milestone; later versions can add feature-engineering, modeling, validation, and interview-defense sections.

### Recommended first exercise

Answer Questions 1, 2, 3, 8, 10, and 12 without copying the model answers. Evaluate conceptual correctness, scientific precision, professional communication, missing reasoning, and resistance to technical follow-up questions.

## Source Basis and Current Scope

This guide reflects the current project stage represented by the following notebooks:

- 01\_preprocessing.ipynb
- 02\_eda.ipynb
- 03\_anomaly\_context.ipynb

The focus is project understanding and scientific reasoning. The project is still in progress, so model-defense and recruiter-interview sections are intentionally limited at this stage.

## 1. Current Project Understanding

### Professional project description

FAHM is a condition-monitoring and anomaly-detection project for the Air Production Unit of a metro train. It uses multivariate time-series sensor data to learn normal machine behavior and eventually detect degradation or failures early. The current work establishes a trustworthy healthy reference by validating the data, understanding the machine's operating states, investigating documented air-leak failures, and separating genuine degradation from maintenance activity, recording gaps, and instrumentation faults.

The intended modeling design currently appears to be:

- Train an anomaly detector on clean healthy operation.
- Do not use failure windows as model inputs.
- Evaluate whether the model raises useful warnings before and during documented failures.
- Separately test whether it detects the verified April degraded period.

This formulation is closer to semi-supervised or one-class anomaly detection than ordinary supervised classification.

## 2. What Is Already Professionally Strong

### 2.1 Pipeline structure

The stages have clear responsibilities:

1. Preprocessing creates a typed and validated data artifact.
2. EDA identifies machine states and sensor relationships.
3. Anomaly context investigates failures and builds reliable analysis labels.
4. Feature engineering will convert those findings into model inputs.
5. Modeling will learn healthy behavior and evaluate early-warning capability.

The rule that later notebooks consume the processed Parquet file rather than repeating preprocessing is a good example of a data contract.

### 2.2 Physical reasoning

The analysis does not treat unusual values as errors automatically. It asks whether several sensors tell a physically consistent story.

- Motor\_current approximately 0 A: motor off.
- Motor\_current approximately 3.9 A: motor running offloaded.
- Motor\_current approximately 5.5-6 A: compressor loaded.
- COMP and DV\_electric are normally complementary.
- TP3 and Reservoirs measure almost the same connected pressure volume.
- High compressor duty combined with elevated oil temperature supports a leak interpretation.

### 2.3 Scientific correction of initial hypotheses

The gaps hypothesis is a strong example of allowing evidence to reject an appealing explanation. A large gap occurred close to F3's repair, so it was tempting to conclude that recording usually stops around maintenance. The base-rate analysis instead showed that gaps are frequent in general, gaps near failures are not generally larger, and most large gaps are far from documented events. The notebook therefore rejects the global hypothesis rather than forcing the data to support it.

### 2.4 Recognition of heterogeneous failures

The four failure windows do not share one common precursor:

- F4: strong rising-duty, motor-current, and oil-temperature signal.
- F1: inverse signal - lower duty and lower temperature before a step-like failure.
- F2: weak oil-temperature signal.
- F3: no meaningful raw central-tendency precursor at 48 or 168 hours.

This is a central finding: a pooled average hides the structure, while per-failure analysis reveals it.

## 3. Most Important Scientific Risks

### 3.1 Only four failure events

The dataset contains approximately 1.5 million rows, but it does not contain 1.5 million independent examples of failure. There are only four documented events, all described as air leaks. The large row count comes from repeated, strongly autocorrelated measurements of the same machine.

- Row-level statistical significance can be misleading.
- Train/test splitting must be temporal.
- Evaluation must be event-level.
- Claims about generalization to other machines or failure types must be modest.

### 3.2 Human-created labels

The categories healthy, prefail, infail, postrepair, degraded, and invalid are useful, but some depend on the project investigation rather than independent ground truth. The April degraded interval was identified partly through elevated duty. If a future detector also uses duty, detecting this interval is useful but is not fully independent validation. It should be described as a corroborating case study, not as equivalent to a separately documented failure label.

### 3.3 Careful wording for F3

#### Supported conclusion

F3 has no detectable precursor in raw-sensor central tendency at the tested 48-hour and 168-hour horizons.

#### Conclusion not yet supported

F3 is impossible to predict.

Variability, slopes, cycle structure, change points, or state-transition features have not yet been exhausted.

### 3.4 Temporal missingness

There are zero NaN values, but the data is not complete in time. The 331 recording gaps represent temporal missingness. Feature engineering should therefore use time-based rolling windows, avoid blind interpolation across long gaps, measure window coverage, and invalidate or flag features when insufficient history exists.

## 4. Questionnaire - Guided Examples

For each guided example, attempt your own answer first. Then compare it with the model answer and identify missing reasoning.

## 1. What problem is the project solving?

### Model answer

The project aims to detect abnormal or degrading behavior in a metro train's Air Production Unit using multivariate sensor data. Because there are only four documented failures and no row-level target column, I am preparing a clean representation of healthy behavior rather than treating it as an ordinary supervised classification problem. The future detector should learn normal machine operation and produce warnings when pressure, temperature, motor load, compressor duty, or cycle behavior deviates in a physically meaningful way. The documented failure windows will mainly be used for event-level evaluation and early-warning analysis.

What this answer demonstrates:

- The physical problem.
- The limited labels.
- The chosen machine-learning formulation.
- The distinction between training and evaluation.

## 2. Why are 1.5 million rows not 1.5 million independent samples?

### Model answer

Measurements are taken approximately every ten seconds from the same machine, so neighboring observations are strongly autocorrelated. A pressure or temperature value at one timestamp is highly dependent on the values immediately before it. In addition, all failure rows belong to only four events. Therefore, the effective amount of independent failure information is closer to four events than hundreds of thousands of failure-related rows. Random row splitting would leak nearly identical operating periods between training and testing and produce overly optimistic results.

Key concept: pseudoreplication - many measurements do not necessarily mean many independent experimental units.

## 3. How can the dataset have no missing values but still have missing data?

### Model answer

Every recorded row has a value in every sensor column, so there are no cell-level missing values or NaNs. However, the timestamp differences reveal intervals where nothing was recorded. Those gaps are missing periods rather than missing cells. They matter because a rolling window may appear to contain one hour of observations while actually spanning several hours because of a recording gap. Feature engineering must therefore use timestamps, track window coverage, and avoid connecting measurements across long gaps as though the machine operated continuously.

#### 4. Why did you avoid relying on a standard t-test?

##### Model answer

A conventional t-test assumes observations are sufficiently independent. Here, ten-second measurements are heavily autocorrelated, and the enormous number of rows would make tiny, operationally meaningless differences appear statistically significant. I therefore focus on robust effect sizes, distribution overlap, physical consistency across sensors, and per-failure behavior. Later, the meaningful test will be whether the detector warns before separate failure events without producing an unacceptable false-alarm rate.

Possible future improvements: event-level comparisons, block bootstrap methods, or time-series-aware uncertainty estimates.

### 5. Questionnaire - Independent Practice

Answer these in your own words. Use evidence and terminology from the project rather than generic machine-learning definitions.

#### 5. What exactly does one row represent?

- An instantaneous sensor snapshot.
- Approximately ten-second spacing.
- Jitter between roughly 9 and 13 seconds.
- Why the values are not ten-second averages.
- What kinds of phenomena become invisible after decimation.

Your answer:

---

#### 6. Why was the original index column removed?

- Why Unnamed: 0 is not a machine feature.
- How it helped reveal decimation.
- Why leaving it in the dataset could create a misleading time proxy.
- Why the real timestamp should determine temporal order.

Your answer:

### **7. Why was converting digital columns to int8 justified?**

- All eight digital columns contained only 0 and 1.
- The full-data purity check.
- The memory benefit.
- Why conversion without validation would have been unsafe.

Your answer:

---

### **8. Explain the physical machine states using TP2, H1, COMP, DV\_electric, and motor current.**

- Motor off.
- Offloaded running.
- Loaded compression.
- How TP2 and H1 change between these states.
- Why multimodal histograms are expected rather than automatically problematic.

Your answer:

---

### **9. Why are TP3 and Reservoirs both redundant and useful?**

- Predictive redundancy: they contain nearly identical pressure information.
- Instrument-health value: disagreement may reveal sensor or circuit problems.
- Why dropping one for modeling does not necessarily mean removing it from monitoring.

Your answer:

### **10. How did you distinguish the April 20 episode from real valve chatter?**

- DV\_electric toggle rate near 99 per hour versus about 3.9 normally.
- Collapse of the normal COMP/DV\_electric antiphase relationship.
- Frozen motor current and TP3.
- Zero or near-zero local variance.
- Why simultaneous digital and analog inconsistency indicates logging failure.

Your answer:

---

### **11. Why is invalid different from degraded?**

- Degraded: measurements are trustworthy and show abnormal machine behavior.
- Invalid: the sensor or logging system is untrustworthy, so the actual machine state cannot be inferred.
- Why neither should be included in a clean healthy training reference.

Your answer:

---

### **12. Why does the pooled prefailure comparison hide useful information?**

- F1 and F4 move in opposite directions.
- Averaging opposite effects can produce a near-zero global effect.
- Failure heterogeneity requires per-event evaluation.
- A detector must recognize abnormality in more than one direction.

Your answer:



---

**13. What feature families follow directly from the current findings?**

- Compressor duty over multiple windows.
- Time spent in off, offloaded, and loaded motor-current modes.
- Oil-temperature level and slope.
- TP3 pressure-decay behavior.
- Cycle frequency and cycle duration.
- Idle-duration features.
- Variability and rate-of-change features.
- Local antiphase share and sensor-stuck indicators.
- Feature-window coverage after recording gaps.

Your answer:

---

**14. How should the final model be evaluated?**

- Whether each failure was detected.
- Warning lead time.
- False alarms per day or week.
- Duration of unnecessary warning states.
- Performance separately for F1, F2, F3, and F4.
- Whether the verified degraded period is detected.
- Temporal holdout rather than random row splitting.
- Why accuracy, ROC-AUC, or row-level recall alone are insufficient.

Your answer:

## 6. Self-Review Rubric

Score each answer from 0 to 2 in every category.

| Criterion                  | 0 - Weak              | 1 - Partial              | 2 - Strong                                             |
|----------------------------|-----------------------|--------------------------|--------------------------------------------------------|
| Conceptual correctness     | Incorrect or confused | Mostly correct with gaps | Correct and internally consistent                      |
| Project evidence           | Generic answer        | Some project details     | Uses specific sensors, events, or findings             |
| Scientific precision       | Overclaims            | Mostly careful           | Clearly separates evidence, inference, and uncertainty |
| Professional communication | Unclear or rambling   | Understandable           | Concise, structured, and defensible                    |
| Follow-up resistance       | Fails basic challenge | Handles one follow-up    | Explains assumptions, limitations, and alternatives    |

## 7. Next Project Milestones

As the project develops, the next versions of this guide can add:

- Feature-engineering rationale and leakage checks.
- Model-selection questions and baseline comparisons.
- Time-series validation design.
- Threshold selection and alarm-policy design.
- Event-level performance analysis.
- Recruiter, data-scientist, and machine-learning-engineer interview simulations.

### Working principle

Each new project step should be understood at three levels: what the code does, why the method is scientifically justified, and how the decision would be defended professionally.

## 8. Target Role Alignment and Learning Plan

This section compares the predictive-maintenance Machine Learning Engineer role supplied by the project owner with the evidence currently visible in the preprocessing, EDA, and anomaly-context notebooks. It distinguishes what the project already demonstrates from what still needs to be implemented or learned.

### Bottom line

This is an excellent project choice for the target role. The domain, data problems, and scientific decisions are closely aligned with predictive maintenance. At the current stage, the project demonstrates a strong data-understanding and investigation foundation, but it is not yet a complete Machine Learning Engineering project because feature extraction, model training, operational evaluation, reproducible inference, and an engineering-facing tool still need to be built.

### Professional summary

Excellent domain and scientific foundation; incomplete end-to-end ML engineering implementation. The goal should be one scientifically defensible system that moves from raw sensor ingestion to trustworthy alerts, rather than adding many technologies without a clear purpose.

### 8.1 Requirement-by-Requirement Comparison

| Role requirement             | Current project evidence                                                                                                | Status                    |
|------------------------------|-------------------------------------------------------------------------------------------------------------------------|---------------------------|
| Messy real-world sensor data | Timestamp jitter, recording gaps, imperfect maintenance records, unusual operating regimes, and instrumentation faults. | Strong                    |
| Data ingestion               | Configuration-based loading, type validation, memory reduction, and a processed Parquet artifact.                       | Good foundation           |
| Feature extraction           | Feature families are scientifically identified, but a reproducible feature pipeline is not yet implemented.             | Next major stage          |
| Anomaly detection            | A healthy-reference strategy is designed, but no detector has yet been trained and compared.                            | Partial                   |
| Degradation detection        | The April degraded interval is investigated and separated from healthy data; no degradation detector exists yet.        | Strong analysis; no model |

| Role requirement                   | Current project evidence                                                                                                                                | Status                  |
|------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------|
| Model training                     | No completed training and selection workflow is visible in the current notebooks.                                                                       | Missing                 |
| Model evaluation                   | The project has good event-level evaluation reasoning, but no actual model results or alert metrics yet.                                                | Partial                 |
| Noisy or partially labeled data    | Only four documented failures, approximate event records, human-created analysis labels, and invalid-data periods are handled explicitly.               | Strong                  |
| Workflow beyond notebooks          | Reusable fahm modules and configuration imports indicate modularization, but tests, CLI execution, and production-style artifacts are not yet verified. | Promising; incomplete   |
| Practical engineering tool         | No dashboard, API, alert interface, or batch inference workflow is currently demonstrated.                                                              | Missing                 |
| Technical communication            | The notebooks document assumptions, rejected hypotheses, limitations, and physical interpretations clearly.                                             | Strong                  |
| Signal processing                  | Temporal, state, and cycle reasoning is present; formal signal-processing methods remain limited.                                                       | Partial                 |
| Diagnostics, prognostics, or RUL   | The project covers anomaly and degradation context, but not diagnosis by fault type or remaining useful life.                                           | Not currently covered   |
| MLOps                              | No verified experiment tracking, model registry, CI, deployment monitoring, or artifact versioning.                                                     | Missing                 |
| Dashboard, API, or edge deployment | Not yet implemented; this is a nice-to-have rather than the first priority.                                                                             | Missing; optional later |

## 8.2 What the Project Already Demonstrates Well

### Messy-data judgment

The project does not reduce data quality to a NaN check. It distinguishes recording gaps, faulty instrumentation, genuine degradation, documented failures, maintenance effects, and rare operating regimes. This directly matches the role statement that models are only part of the job.

### Physical interpretation

The analysis explains loaded, offloaded, and off motor states; pressure behavior; valve relationships; compressor duty; and oil temperature. This shows the ability to treat sensors as measurements of a physical system rather than anonymous numerical columns.

### Partially labeled learning problem

The project recognizes that millions of timestamps do not create millions of independent failure examples. The four documented failures mainly support event-level evaluation, while clean healthy operation can support one-class or semi-supervised anomaly detection.

### Scientific restraint

The notebooks separate supported conclusions from attractive but unproven explanations. They reject the global gap-maintenance hypothesis and avoid claiming that F3 is unpredictable merely because raw central tendencies did not reveal a precursor.

## 8.3 Immediate Learning Priority: Time-Series Feature Engineering

The next notebook should turn the current physical findings into a reproducible, time-indexed feature table. Every feature must use only information available at or before its prediction timestamp.

### Machine-state features

- Fraction of time the motor is off, offloaded, and loaded.
- Compressor duty over multiple historical windows.
- Longest continuous idle period and time since the last loaded cycle.
- Number and rate of state transitions.

These are often more meaningful than mean motor current because the current distribution is multimodal. A mean may represent a mixture of states rather than a real operating state.

### Trend and variability features

- Mean, median, standard deviation, robust IQR, minimum, and maximum.
- Linear slope, rate of change, and difference from an earlier baseline.
- Exponentially weighted moving statistics.
- Rolling robust z-scores or residuals relative to healthy behavior.
- Multiple look-back windows such as 15 minutes, 1 hour, 6 hours, 24 hours, 48 hours, and 168 hours when physically justified.

### Cycle features

- Cycle duration and cycles per hour.
- Loaded and offloaded durations.
- Pressure increase during compression and pressure decay while not compressing.
- Time required to recover pressure.
- Variability and change in cycle structure.

Because the compressor naturally operates in cycles, cycle-level behavior may be more informative and more interpretable than isolated raw readings.

### Cross-sensor and data-quality features

- TP3 minus Reservoirs as a sensor-consistency measure.
- Local COMP and DV\_electric antiphase share.
- Motor current conditioned on valve state.
- Pressure response conditioned on compressor state.
- Oil-temperature response to compressor duty.
- Window coverage, largest internal gap, timestamp regularity, frozen-value indicators, and low-variance flags.

### Two-stage monitoring principle

Data-quality assessment should precede machine-health interpretation. A machine-health detector should not treat an invalid sensor window as trustworthy evidence of mechanical degradation.

## 8.4 Time-Aware Validation

Random row splitting is not acceptable because adjacent ten-second measurements are strongly autocorrelated and would leak nearly identical operating periods between training and testing. Preprocessing parameters, scalers, feature baselines, and thresholds must be fitted only on historical training data.

Methods to learn and apply:

- Chronological train-validation-test splitting.
- Walk-forward or expanding-window validation.
- Blocked temporal cross-validation.
- Event-based evaluation for the four documented failures.
- Threshold calibration on a separate healthy period.
- Per-event reporting rather than pooled row-level performance.

### Realistic experiment pattern

Train on clean healthy history available before an event; calibrate the alert threshold on a separate historical healthy period; evaluate the upcoming failure; repeat where sufficient prior history exists. With only four failures, generalization uncertainty must remain explicit.

## 8.5 Models and Concepts to Learn

### A small, justified model set

The project does not need many advanced models. It needs interpretable baselines and a fair comparison.

- Engineering-rule baseline: high duty, abnormal pressure decay, long idle periods, sensor antiphase violations, or frozen signals.
- Statistical baseline: robust z-scores, EWMA, CUSUM, Mahalanobis distance, PCA reconstruction error, or change-point detection.

- Machine-learning detector: Isolation Forest, PCA-based anomaly detection, One-Class SVM after careful scaling, or a shallow autoencoder only after simpler baselines.

### Model-complexity principle

The dataset has many timestamps but only four independent documented failures from one machine. A simpler healthy-reference detector may be more defensible than an LSTM or other deep model. Complexity should be earned by better event-level performance, lower false-alarm burden, or clearer operational value.

### Anomaly, degradation, prediction, and RUL

**Anomaly detection:** Identifies behavior that differs from learned normal operation. An anomaly may be a machine problem, sensor fault, maintenance action, or rare valid regime.

**Degradation detection:** Looks for persistent or worsening movement away from healthy condition, often through sustained trends or a health indicator.

**Failure prediction:** Estimates whether a failure will occur within a defined future horizon. Four documented failures are too few for a trustworthy supervised classifier.

**Prognostics or RUL:** Estimates remaining time before failure. The current dataset does not provide many independent run-to-failure trajectories with clear degradation starts, so a defensible RUL model is not supported.

## 8.6 Signal-Processing Knowledge

The role lists signal processing as one relevant background area. For this project, the most useful topics are those that match slow compressor dynamics rather than high-frequency vibration analysis.

- Smoothing and filtering, including the effect of filter choice on delay and peaks.
- Autocorrelation and cross-correlation.
- Change-point detection.
- Cycle-frequency and state-transition analysis.
- Derivatives and rates of change.
- Irregular sampling, resampling, interpolation risk, and aliasing.
- Conceptual understanding of Fourier transforms and power spectral density.

FFT features should not be inserted merely to claim signal-processing experience. At approximately ten-second sampling, physically meaningful cycle and pressure-dynamics features may be more valuable. Frequency-domain methods would become much more central with high-frequency vibration or acoustic sensors.

## 8.7 Engineering Beyond Notebooks

The final repository should allow another person to reproduce preprocessing, feature generation, training, evaluation, and inference without manually executing notebook cells.

### Minimum workflow capabilities

- Reusable source modules rather than duplicated notebook logic.

- Configuration files for paths, windows, thresholds, and model parameters.
- One command or script for preprocessing, feature building, training, evaluation, and prediction.
- Structured logging and useful failure messages.
- Saved preprocessing, feature-schema, threshold, and model artifacts.
- Batch inference on unseen sensor data.
- A clear README describing setup, commands, methodology, and limitations.

### Tests that matter

- Digital signals contain only allowed values.
- Timestamps are sorted and duplicate timestamps are handled explicitly.
- Rolling features do not bridge invalid periods or large gaps incorrectly.
- A feature at time  $t$  never uses measurements after time  $t$ .
- Low-coverage windows are flagged or excluded.
- Training excludes degraded, failure, maintenance, and invalid periods according to the defined policy.
- Inference returns the expected schema.
- Alert aggregation and hysteresis behave correctly.

### Most important test

Prove that every feature calculated at time  $t$  uses only past and present information. A hidden look-ahead error can make an apparently excellent predictive-maintenance model useless in deployment.

## 8.8 Evaluation for Engineering Usefulness

A classification report or ROC-AUC score is not enough. Engineers need stable alert events rather than thousands of isolated red timestamps.

A practical alert policy can include:

1. Calculate an anomaly or health score.
2. Smooth or aggregate the score.
3. Require the score to remain abnormal for a minimum duration.
4. Open an alert only after the persistence condition is met.
5. Use hysteresis so the alert does not repeatedly open and close.
6. Close the alert only after the system remains normal for a defined recovery period.

Report at least:

- Number of documented failure events detected.
- First warning timestamp and lead time for F1, F2, F3, and F4.
- False alerts per day or week.
- Average alert duration and percentage of healthy time under warning.
- Detection of the verified April degraded interval.
- Behavior during the invalid instrumentation period.
- Main features or sensor relationships contributing to each alert.



## 8.9 Practical Tool and MLOps Scope

### Engineering-facing output

A simple Streamlit dashboard or API-backed report is sufficient for a strong portfolio version. It should communicate current health state, data-quality state, active alerts, alert duration, recent sensor timelines, and the main evidence behind an alert.

#### Useful alert wording

Sustained degradation warning: compressor duty is substantially above its healthy baseline, loaded-state occupancy is increasing, and oil temperature has risen for several hours. Sensor-quality checks passed.

This is more useful than reporting only an anomaly score because it translates model output into an engineering investigation.

### MLOps priorities

Necessary for this project:

- Reproducible environment using Poetry or pyproject.toml.
- Configuration management, modular code, tests, logging, and saved artifacts.
- Git version control and reproducible training and inference commands.

Strong additions after the modeling workflow is trustworthy:

- MLflow or another experiment tracker.
- Docker.
- GitHub Actions for automated tests.
- Data or artifact versioning.
- FastAPI inference endpoint and structured prediction logs.

Optional later extensions: cloud deployment, model registry, automated retraining, streaming infrastructure, ONNX conversion, or edge deployment. These should not replace careful feature and validation work.

## 8.10 What Not to Force Into the Project

**RUL prediction:** The role lists RUL as a nice-to-have. The current data does not appear to support reliable remaining-life supervision.

**Deep learning:** A neural network is not automatically more professional. With four failure events, overfitting and weak validation are major risks.

**Fault-type classification:** The event count and apparent concentration on air-leak failures do not support a meaningful multiclass diagnostic model.

**Vibration analysis:** The project has pressure, temperature, and current sensors, which already satisfy the industrial sensor requirement. It should not pretend to contain vibration information.

**Edge deployment:** This is optional. A reproducible local pipeline and useful dashboard are higher-value first milestones.

## 8.11 Recommended Build Order

**Stage 4 - Feature engineering:** Create a versioned feature artifact containing machine-state occupancy, cycle behavior, pressure dynamics, trends, variability, sensor consistency, and coverage indicators.

**Stage 5 - Baselines and anomaly models:** Compare an engineering-rule baseline, a statistical baseline, and one or two justified ML anomaly detectors.

**Stage 6 - Temporal and event-level evaluation:** Implement chronological evaluation, per-failure lead time, false-alert frequency, persistence, hysteresis, and invalid-sensor handling.

**Stage 7 - Production-style workflow:** Move stable logic into source modules and add training, evaluation, inference, tests, logs, stored artifacts, and reproducible commands.

**Stage 8 - Engineering interface:** Build a small dashboard or API that communicates health state, sensor-quality state, alert timeline, and explanation.

## 8.12 Final Assessment

| Dimension                | Assessment       | Reason                                                                                                                    |
|--------------------------|------------------|---------------------------------------------------------------------------------------------------------------------------|
| Topic alignment          | <b>Excellent</b> | The project is directly in predictive maintenance and industrial multivariate time series.                                |
| Scientific foundation    | <b>Strong</b>    | The work carefully distinguishes machine behavior, degradation, data gaps, and instrumentation problems.                  |
| Current ML completeness  | <b>Early</b>     | No completed feature pipeline, trained detector, threshold calibration, or model comparison is visible yet.               |
| Current MLE completeness | <b>Partial</b>   | Modularization is promising, but automated workflows, tests, artifacts, inference, and an engineering-facing tool remain. |

| Dimension           | Assessment       | Reason                                                                                                             |
|---------------------|------------------|--------------------------------------------------------------------------------------------------------------------|
| Portfolio potential | <b>Very high</b> | A completed end-to-end system could demonstrate nearly every central requirement in the supplied role description. |

### Target end-to-end story

Raw sensor ingestion -> data-quality validation -> physical feature engineering -> healthy-reference anomaly and degradation detection -> temporal event-level evaluation -> stable alert policy -> reproducible pipeline -> engineering dashboard.

### Final working principle

Do not try to prove knowledge by adding every method or technology. Build one trustworthy system and be able to explain what the code does, why the method is scientifically justified, how leakage and uncertainty are controlled, and how the result helps an engineer make a decision.